AI in the Loop (AITL): A Systems Taxonomy for Closed-Loop Autonomous Evaluation

Sanskar Jajoo
Independent Researcher, India
https://github.com/m4vic

Abstract

We identify and formalize AI In The Loop (AITL), a paradigm where AI systems autonomously generate, evaluate, and improve without human intervention in operational workflows. AITL extends the RLAIF principle—replacing human feedback with AI feedback—from training to the full AI system lifecycle.

Through analysis of four systems (AlphaZero, Constitutional AI, SWE-agent, autoresearch), we extract common properties and propose a unifying taxonomy: self-generation, self-evaluation, self-improvement, and human observation. We validate AITL through a controlled experiment using the Autonomous Empirical Optimization System (AEOS), a model-agnostic ML sandbox where two LLM agents autonomously built ML pipelines on a semantically-stripped dataset. In our experiment, we observe that the dominant human role in AITL shifts from iterative ML engineering ($O(n)$ per iteration) to boundary supervision ($O(1)$ per experiment).

Our contributions are: (1) formalization of AITL as a unifying framework for closed-loop autonomous systems, (2) a taxonomy connecting existing systems under shared properties, (3) empirical validation via AEOS demonstrating autonomous agent stopping behavior, and (4) identification of failure modes including an explicit Sunk-Cost Continuation failure mode (F6). We position AITL as a natural evolution of AI evaluation, suggesting scalable directions infeasible under HITL constraints.

Keywords: AI evaluation, closed-loop systems, autonomous ML, neural architecture search, self-improving AI

1 Introduction

1.1 The Human Bottleneck

AI systems increasingly require continuous evaluation: safety testing for each model version, adversarial robustness across attack families, regression detection over time, and cross-model comparison.

Traditional Human-in-the-Loop (HITL) evaluation creates bottlenecks:

- Expert reviewers cost $200–500/hour
- Manual testing limits coverage
- Human fatigue reduces consistency
- Doesn't scale to continuous deployment

For example, GPT-4 red-teaming required 50+ external experts over 6 months [OpenAI, 2023]. Each model update requires repeating this expensive process.

1.2 The Pattern We Propose to Unify

Several major AI systems have independently solved this problem by removing humans from operational loops:

AlphaZero (2017): Game evaluation via self-play. No human game records needed—AI generates positions, evaluates outcomes, and improves policy, achieving superhuman performance [Silver et al., 2017].

Constitutional AI (2022): Alignment via RLAIF. AI judges AI outputs against principles with minimal human feedback, enabling scalable alignment [Bai et al., 2022].

autoresearch (Karpathy, 2026): Autonomous ML experimentation. AI modifies code, runs experiments, and evaluates results while humans review only the final output [Karpathy, 2026].

We propose a unifying interpretation of these systems under a common operational framework: **AI In The Loop (AITL)**.

1.3 Contributions

We formalize AI In The Loop (AITL) and provide:

1. **Paradigm Definition**: Clear properties distinguishing AITL from HITL, with formal requirements.
2. **Taxonomy of Existing Systems**: Analysis of AlphaZero, Constitutional AI, SWE-agent, and autoresearch as systems exhibiting AITL-like properties.
3. **Experimental Validation**: A reproducible proof-of-concept demonstrating the self-improving feedback loop.
4. **Failure Mode Analysis**: Identification of when and how AITL systems fail.
5. **Roadmap**: Open challenges and future research directions.

1.4 Scope and Novelty

We note explicitly: the contribution of this work is *not* the invention of closed-loop AI systems—these have existed for decades. Our contribution lies in:

- Identifying a common set of operational properties across independently developed systems
- Formalizing these properties into a reusable taxonomy
- Providing empirical evidence via a controlled experiment
- Defining failure modes and requirements for safe deployment

1.5 Paper Organization

Section 2 provides background on HITL and the RLHF→RLAIF evolution. Section 3 analyzes existing AITL-like systems. Section 4 presents the formal AITL definition and properties. Section 5 examines autoresearch as an external case study. Section 6 describes our controlled experimental validation via AEOS. Section 7 discusses future applications and limitations. Section 8 concludes.

2 Background

2.1 Human-in-the-Loop (HITL) Evaluation

HITL has been the standard for AI system validation. Humans occupy critical decision points in operational workflows, reviewing AI outputs before deployment.

Examples: Content moderation (humans review flagged posts), medical AI (doctors verify diagnoses), autonomous vehicles (safety drivers monitor systems), and model evaluation (red-teamers test for failures).

Advantages:

- Human judgment on edge cases
- Accountability (human made final decision)
- Regulatory compliance in high-stakes domains

Limitations:

- Linear scaling with workload (more tests = more humans)
- Inconsistency (fatigue, subjectivity, drift over time)
- Cost (expert time is expensive, scarce)
- Latency (human review is slow)

2.2 From RLHF to RLAIF: Training's Evolution

Reinforcement Learning from Human Feedback (RLHF) dominated LLM alignment [Christiano et al., 2017, Ouyang et al., 2022]:

1. Generate model outputs
2. Humans rank outputs (A better than B)
3. Train reward model on rankings
4. Optimize policy via RL

Bottleneck: Requires 50K–500K human labels.

Bai et al. [2022] introduced Constitutional AI, using Reinforcement Learning from AI Feedback (RLAIF):

1. Define constitutional principles (human-provided)
2. AI critiques outputs against principles
3. AI ranks critiques
4. Train from AI feedback

Result: 90% reduction in human labeling, comparable alignment quality. **Key insight:** AI can judge AI when given a proper framework.

2.3 The Missing Piece: HITL Evaluation Remains

While RLAIF solved training bottlenecks, evaluation workflows still rely on HITL. *Can we apply RLAIF's lesson to evaluation?* Our results suggest: yes, via AITL.

Figure 1 illustrates this evolution from manual per-instance evaluation (HITL) through training-phase automation (RLHF, RLAIF) to operational-phase automation (AITL), and positions existing systems as bounded instantiations of the AITL paradigm.

2.4 Related Paradigms

AITL intersects with, but differs from, several existing concepts:

AutoML / NAS. Systems like Auto-sklearn [Feurer et al., 2015], TPOT [Olson et al., 2016], DARTS [Liu et al., 2019], and ENAS [Pham et al., 2018] focus on architecture or hyperparameter search. AITL is broader: any autonomous generate-evaluate-improve loop, not limited to model selection.

Active Learning. Humans selectively label the most informative examples [Settles, 2009]. AITL removes humans from the labeling loop entirely—the evaluator is itself an AI component.

Self-Play. AlphaZero [Silver et al., 2017] and related systems use self-play in adversarial game settings. AITL generalizes beyond zero-sum games to any evaluation domain.

Meta-Learning. MAML [Finn et al., 2017] and related approaches learn how to learn across task distributions. AITL learns what works via feedback on a specific task instance.

LLM Agents. Systems like AutoGPT [Richards, 2023] and BabyAGI [Nakajima, 2023] use LLMs for autonomous task execution. These represent instances of AITL when they incorporate evaluation feedback loops, though many lack the self-improving property (P3).

We propose AITL as a unifying framework that encompasses these as special cases, distinguished by the four properties defined in Section 4.

3 Existing AITL-Like Systems

We analyze foundational systems that independently exhibit AITL properties, acting as precursors to the paradigm. However, we note that none of these constitute fully autonomous closed-loop instances under our stricter definition (Section 4), as each is restricted in domain scope, evaluation dynamics, or termination autonomy.

3.1 AlphaZero: Closed-Loop Game Mastery (2017)

AlphaZero [Silver et al., 2017] exhibits strong AITL-like closed-loop properties:

- *Self-Generating*: Generates game positions through self-play
- *Self-Evaluating*: MCTS search evaluates position quality
- *Self-Improving*: Policy network updated from game outcomes
- *Human-Observed*: Researchers monitor Elo ratings

Bounded instantiation: AlphaZero is narrowly scoped to deterministic, perfect-information, zero-sum games where the evaluation function (win/loss) is mathematically guaranteed. It cannot function in open-ended operational environments where evaluation criteria are abstract or ambiguous.

3.2 Constitutional AI: Closed-Loop Alignment (2022)

Bai et al. [2022] demonstrated AITL properties for language alignment:

- *Self-Generating*: AI generates response critiques
- *Self-Evaluating*: AI judges outputs against constitutional principles
- *Self-Improving*: RLAIF training updates model behavior
- *Human-Observed*: Humans define principles, review aggregate metrics

Bounded instantiation: Constitutional AI applies the AITL philosophy exclusively to the *offline pre-training and fine-tuning* phase. It is not a real-time operational agent capable of independent exploration or dynamic constraint management.

The system achieved comparable alignment quality to RLHF with 90% fewer human annotations, suggesting that AI judgment can substitute for human judgment when given appropriate constraints.

3.3 autoresearch: Closed-Loop ML Experimentation (2026)

Karpathy [2026] released autoresearch, a public framework where AI agents autonomously conduct ML experiments:

- *Self-Generating*: Agent modifies `train.py` with new hyperparameters, architectures, and optimization strategies

Table 1: Cross-system comparison of AITL properties across four foundational systems. All exhibit core AITL properties but with distinct autonomy bounds.

Property	**AlphaZero**	**Constitutional AI**	**autoresearch**	**SWE-agent**
Self-Gen	Game positions	Response critiques	Code modifications	Code patches
Self-Eval	MCTS value	Constitutional judgment	`val_bpb` metric	Unit test pass/fail
Self-Improve	Policy update	RLAIF training	Git commit/revert	Iterative patch
Human Role	Monitor Elo	Define principles	Set goals	Define test suite
Autonomy bound	Game-only	Offline phase	No self-stop	Human test harness

- *Self-Evaluating*: Validation bits-per-byte (`val_bpb`) provides objective scoring after each 5-minute training run
- *Self-Improving*: Agent uses git-integrated history of successes and failures to inform subsequent experiments
- *Human-Observed*: Researcher defines goals in `program.md`, reviews final committed improvements

When pointed at nanochat (a well-tuned GPT-2 training codebase), the agent ran approximately 700 experiments over two days, identifying multiple improvements missed by human developers. These changes reduced time-to-GPT-2 from 2.02 to 1.80 hours—an 11% improvement on an already heavily optimized baseline.

3.4 SWE-agent: Closed-Loop Software Engineering (2024)

Jimenez et al. [2024] introduced SWE-bench and the accompanying SWE-agent—an LLM agent that autonomously resolves real GitHub issues:

- *Self-Generating*: Agent writes code patches and unit test commands autonomously, exploring the codebase without human guidance
- *Self-Evaluating*: Existing unit test suites provide an objective pass/fail signal after each patch attempt
- *Self-Improving*: The agent integrates test failure messages into subsequent patch attempts, iterating toward a passing state
- *Human-Observed*: Humans define the benchmark; the agent resolves issues independently

Bounded instantiation: SWE-agent's evaluation signal comes from a pre-written human test suite. The agent cannot author *new* tests to verify its own solutions against uncovered corner cases. Self-evaluation is therefore bounded by the quality of the human-authored harness.

3.5 Cross-System Comparison

Table 1 summarizes the cross-system comparison. These systems succeed when: (1) success metrics are clearly definable, (2) AI can generate meaningful variations, (3) AI can judge quality against the metric, and (4) a feedback loop drives iterative improvement.

4 AITL: Formal Definition

4.1 Definition

AI In The Loop (AITL): A system architecture where AI components autonomously generate inputs, evaluate outputs, and improve behavior through feedback loops, with humans relegated to

observation and periodic calibration rather than operational decision-making.

4.2 Formal Model

We define the AITL loop as a discrete-time dynamical system:

$$S_{t+1} = U\big(S_t,\ E\big(G(S_t)\big)\big) \tag{1}$$

This form separates AITL from conventional automation: evaluation itself is endogenous to the loop rather than an external human gate. Note that E may itself be learned, fixed, or ensemble-based—the evaluator architecture is a key design variable in any AITL instance.

Where:

- S_t is the system state at iteration t
- G (Generator): Produces candidate outputs from current state
- E (Evaluator): Scores candidates against success criteria
- U (Updater): Modifies system state based on evaluation

An AITL system satisfies the following:

1. G, E, and U operate **without** per-instance human input
2. The loop is bounded: $\exists T$ such that $\|S_{T+1} - S_T\| < \varepsilon$ (convergence) or a human kill switch is triggered
3. Human role is limited to: defining G's domain, calibrating E's criteria, and monitoring aggregate metrics of U

4.3 Core Properties

An AITL system must satisfy four properties:

P1. Self-Generating. AI creates test inputs, prompts, or scenarios without human authorship for each instance. Formally: G operates autonomously; human provides only the initial domain specification.

P2. Self-Evaluating. AI judges output quality with quantified confidence, without human labeling per instance. Formally: E produces a scalar score without human annotation.

P3. Self-Improving. Feedback from evaluation drives system adaptation without manual human intervention. Formally: U updates S based solely on E's output.

P4. Human-Observed. Humans monitor aggregate metrics, audit periodically, and manage system constraints (e.g., API budgets, execution timeouts, safety kill-switches), but do not operate the loop continuously. Formally: Human interaction is $O(1)$ per experiment (defining success criteria, setting financial/compute bounds), rather than $O(n)$ per iteration. The human transitions from ML developer to system manager.

4.4 Requirements for AITL Success

Based on analysis of existing systems, AITL requires:

R1. Measurable Success Criterion: Clear metric for "good" vs. "bad" (e.g., validation loss, Elo rating, `val_bpb`).

R2. Reliable AI Judgment: Self-evaluation must correlate with ground truth. E's scoring function must be robust to optimization pressure.

R3. Bounded Feedback Loop: System must converge or plateau, not diverge. Safeguards prevent runaway optimization.

R4. Audit Mechanism: Human inspection of samples, metrics, and behavior to detect drift or misalignment.

R5. Kill Switch: Humans can halt the system if metrics degrade.

4.5 When AITL Fails: Known Failure Modes

Based on existing systems and our experiments, AITL is susceptible to the following failure modes:

F1. Objective Misspecification. System optimizes a proxy metric ("judge says safe") instead of the true objective ("actually safe"). The agent learns to satisfy the evaluator without achieving the intended goal.

F2. Evaluator Collapse. When generator and evaluator are coupled, the evaluator may gradually accept lower-quality outputs, leading to drift. This risk is well-documented in LLM-as-judge settings [Wang et al., 2023b, Zheng et al., 2023].

F3. Feedback Ambiguity. In domains where "good" is subjective (creative writing, UX design), AI judges may disagree, and the feedback loop may not converge.

F4. Insufficient Exploration / Local Attractors. System gets stuck in local optima. Once a model family achieves a good result, the agent may over-exploit it rather than exploring alternative families that could yield superior performance.

F5. Adversarial Environment. When optimizing against an adaptive adversary (spam detection vs. evolving spammers), the environment changes faster than the AITL loop can adapt.

F6. Sunk-Cost Continuation. The agent continues proposing low-yield variations despite prolonged metric stagnation and absent evidence of expected improvement, requiring external budget or policy constraints to terminate execution. F6 differs from F4 (local attractors) because the agent is not trapped in a suboptimal region—it has already found a near-optimal solution but cannot recognize that further exploration is unwarranted. This was empirically observed in our AEOS experiment (Section 6.4).

Mitigation strategies include: ensemble evaluators (F1, F2), explicit exploration-exploitation balance mechanisms (F4), hard stagnation cutoffs (F6), and periodic human recalibration of evaluation criteria (F3, F5).

Concrete failure examples from AEOS (Section 6).

- **F6 (Sunk-Cost Continuation):** GPT-4o-mini peaked at 80.90% accuracy at iteration 8. Despite explicit budget-conservation instructions, it continued for 28 more iterations exploring MLP variants (e.g., `MLPClassifier(128,64)`, `MLPClassifier(100,50)`) and complex voting ensembles—none of which could statistically beat RandomForest on this tree-structured tabular dataset.
- **F4 (Local Attractor):** GPT-4o-mini converged on RandomForest + VotingClassifier variants as a dominant strategy and repeatedly recycled the same ensemble combinations with minor parameter tweaks across 15+ iterations.
- **F3 (Feedback Ambiguity Prevention):** Qwen2.5-Coder avoided F3 entirely by issuing the `STOP` signal autonomously—it correctly recognized that its plateau at ∼80.65% with RandomForest was stable and terminated rather than exploring noisily.

Table 2: Comparison of HITL and AITL evaluation paradigms.

Aspect	HITL	AITL
Input Generation	Human authors each test	AI generates tests
Evaluation	Human judges each output	AI judges with confidence
Improvement	Human updates manually	Automated feedback loop
Scaling	Linear with humans	Limited by compute only
Cost	High (expert time)	Low (API/compute)
Consistency	Variable (fatigue)	Deterministic per config

4.6 AITL vs. HITL Comparison

5 External Case Study: autoresearch

A public repository released by Andrej Karpathy documents autonomous experimentation loops under the name autoresearch [Karpathy, 2026]. We examine this system to validate AITL properties in a real-world scenario not designed by the present authors.

Experimental Setup:

- Base system: nanochat—a well-tuned GPT-2 training codebase
- Goal: Reduce training time to reach GPT-2 baseline perplexity
- Agent: AI coding agent with code execution capabilities
- Scope: Agent may only modify `train.py`; evaluation harness is immutable to prevent metric gaming
- Duration: ∼2-day autonomous run, approximately 700 experiments
- Metric: Validation bits-per-byte (`val_bpb`)

AITL Properties Demonstrated:

Self-Generating: Agent autonomously modified PyTorch initialization schemes, weight decay schedules, attention mechanisms (QKNorm multipliers, banded attention), and optimization hyperparameters (AdamW betas, weight decay) without human templates.

Self-Evaluating: Agent judged success via `val_bpb` after each fixed 5-minute training run. Changes that improved the metric were git-committed; changes that degraded it were git-reverted.

Self-Improving: The sequence of ∼700 experiments informed subsequent hypotheses. The agent reported multiple implementation improvements over baseline runs across diverse optimization axes.

Human-Observed: The researcher defined the objective in `program.md`, then reviewed the final aggregate results. No intervention occurred during the ∼700-experiment run.

Result: Time-to-GPT-2 reduced from 2.02 to 1.80 hours—an 11% improvement on an already heavily optimized baseline. Subsequent community iterations further reduced this to ∼1.65 hours.

This validates AITL feasibility for complex, multi-step research tasks. However, autoresearch does not constitute a fully autonomous closed-loop instance under our definition. The evaluation harness is hard-coded and the script runs indefinitely until a human manually kills the terminal—the AI does not judge its own completion or manage resource constraints. It lacks the critical self-terminating feedback mechanism required for constraint-aware autonomous operation.

6 Controlled Experimental Validation: AEOS

6.1 The Autonomous Empirical Optimization System (AEOS)

Before describing the experiment, consider what AITL replaces in a concrete engineering workflow.

Table 3: AEOS experiment specifications.

Parameter	Value
Task	Tabular multi-class classification
Metric	Validation Accuracy
Agent Autonomy	Full (Model Family, Preprocessing, Hyperparameters, Stopping)
Hard Stagnation Cutoff	3 consecutive pivot loops (15 iterations without improvement)
Hardware Timeout	120 seconds execution cap per iteration

The HITL scenario: A human ML engineer is given an unknown dataset and asked to build the best possible classifier. They explore data manually, research architectures, test baselines, implement changes, and interpret results in a loop that takes hours to days.

The AITL scenario: The human defines the dataset, budget, and safety constraints (Property P4: Human-Observed). The AI agent is assigned to maximize accuracy autonomously.

To study a constrained autonomous closed-loop evaluation instance and address the missing termination autonomy observed in systems like autoresearch, we built the Autonomous Empirical Optimization System (AEOS)—a model-agnostic ML sandbox. In AEOS, the agent writes arbitrary Python model training code (scikit-learn, PyTorch, etc.) *and* must evaluate its own performance plateaus to autonomously output a `STOP` command when further improvement is unlikely.

6.2 Experimental Setup: The Blind Dataset

To ensure agents cannot simply recall known-good parameters from pre-training data based on semantic context, we utilized a real-world dataset (Cover Type from UCI) with all semantic markers stripped. Features were renamed to integers, and labels were integer-mapped, reducing the problem to raw statistical discovery.

Information Withheld from Agent: The LLM is told only: `n_features=54`, `n_classes=7`, `n_train=8000`, `n_val=2000`. No dataset name, no domain, and no feature descriptions are provided.

6.3 The Loop Architecture

6.4 Results: Two Agent Comparison

To validate the robustness of the system across different AI backends, we ran the exact same AITL environment using a local model designed for coding versus a commercial, generalist cloud model.

Agent 1: Qwen2.5-Coder-7B (Local System).

- **Baseline (Iteration 1):** Autonomously identified `RandomForest` as the optimal baseline family for obscure tabular data, achieving **80.65%** accuracy.
- **Exploration:** Systematically tested scaling and hyperparameters.
- **Stopping Behavior:** After 9 iterations and a slight pivot to test non-tree structures, it inferred from recent metric history that further gains were unlikely. It output the `STOP` signal autonomously.
- **Cost:** $0 (local execution).

Agent 2: GPT-4o-mini (Cloud System).

- **Exploration:** Broadly explored the model space, testing `GradientBoosting`, `RandomForest`, `SVM`, `sklearn_MLP`, and `ExtraTrees`.
- **Optimal Discovery:** Peaked at **80.90%** at Iteration 8.
- **Stopping Behavior:** Despite explicit budget-conservation instructions, the agent continued low-yield exploration for 28 iterations after the best score at iteration 8, generating marginal pipeline variants without emitting a `STOP` signal. No stop signal was observed within the available budget window. The run was externally terminated at iteration 36 due to API budget constraints.
- **Cost:** ∼$1 under then-current API pricing and token budget, expended on negligible marginal gains.

6.5 Analysis and Generalization

This validation confirms the AITL properties:

1. **Self-Generating / Evaluating:** Both agents wrote complete pipelines, evaluated metrics, and integrated feedback autonomously without a human writing a single line of machine learning code.
2. **The "Human Observer" Pivot:** Moving the human from a coding role to a constraint-management role proved critical. While GPT-4o-mini found a slightly higher global best (80.90% vs. 80.65%), no stop signal was observed within the available budget window, emphasizing P4's necessity. The cloud agent expended 28 iterations on `MLPClassifier` variants—statistically inappropriate for this tree-structured tabular dataset—and ensemble combinations it had already effectively tried, consistent with F6 (Sunk-Cost Continuation). This highlights that practical AITL deployments remain constrained not only by convergence logic but also by economic resource policies.
3. **Architecture as a Data Property:** The local agent never tried MLP once. This pragmatic choice was empirically correct—Cover Type data has discrete categorical splits (soil type, wilderness area) that tree-based splits exploit natively. The cloud model's broader exploratory behavior likely reflects general-purpose instruction tuning encouraging exhaustive search regardless of data suitability.

Note on statistical validity: Results are from a single representative run of each agent. Variance across multiple seeds is left to future work; this experiment serves as a qualitative behavioral study of autonomous stopping strategies.

6.6 The Comparative Frontier

Figure 3 plots the comparative Best-So-Far discovery frontier between the two agents. The visual disparity illustrates the behavioral difference under identical AITL constraints: the local model self-terminated upon reaching a performance plateau, while the cloud model continued exploration without emitting a stop signal for 28 additional iterations beyond its peak.

7 Future Applications and Limitations

7.1 Continuous Safety Evaluation (AITL-Safety)

We propose an architecture for continuous safety evaluation:

- *Attack Generator (Self-Gen):* Template-based jailbreaks (50+ patterns), mutation strategies, adversarial suffix generation

- *Target Executor:* Multi-model testing (GPT, Claude, Llama, etc.), parallel execution (100+ attacks/hour)
- *Judge Ensemble (Self-Eval):* Frozen baseline judge (never updated, prevents drift), adaptive judge, cross-model judges for tie-breaking
- *Regression Engine (Self-Improve):* Safety delta monitoring, alert if delta exceeds threshold, retrain adaptive judge on new attack families
- *Human Observer:* Weekly dashboard review, audit 50 random samples/month, kill switch if attack success rate spikes

Estimated impact: Current HITL manual red-teaming costs ∼$200/hr at ∼5 attacks/hr. AITL-Safety: ∼$0.15/attack at ∼100 attacks/hr—estimated cost reduction of ∼250×, throughput increase of ∼20×.

7.2 Autonomous Code Review

AI agents can generate edge-case unit tests (Self-Generating), execute them against pull requests (Self-Evaluating), and suggest code modifications based on failures (Self-Improving), while humans review only the final aggregate report (Human-Observed). This is already partially realized in tools like GitHub Copilot's automated PR review, though these systems currently lack the full closed-loop self-improvement property.

7.3 Scientific Peer Review

Scaling scientific literature review requires an AITL approach where AI systems flag methodological errors or literature gaps autonomously, enabling human reviewers to focus on high-level conceptual judgments. This application carries significant risk of F1 (Objective Misspecification) if the evaluator optimizes for surface-level writing quality rather than scientific rigor.

7.4 Limitations of This Work

Experiment scope: Our AEOS experiment validates P3 (Self-Improving) on a single synthetic classification task. Broader validation across domains (NLP, computer vision, reinforcement learning) is required to claim generality.

Agent capabilities: Results depend on GPT-4o-mini's code generation quality. Weaker models may require more iterations or fail to converge entirely.

Memorization vs. Discovery: While we provide evidence of functional discovery through the feedback loop (Section 6), we cannot definitively prove the agent did not retrieve architectural patterns from pre-training. Stronger validation would require training an agent from scratch on non-public architecture data.

Generalization: AITL succeeds for well-defined optimization problems (game playing, architecture search, training speedup). Applicability to subjective domains (creative evaluation, novel scientific hypotheses) remains an open question.

Cost at scale: While individual experiments are cheap (∼$1), scaling to 10,000+ iterations or using expensive models (GPT-4o) may incur significant costs. Cost-benefit analysis is required per application.

Safety risks: AITL systems optimizing harmful objectives (e.g., maximizing jailbreak success rates) without safeguards could create autonomous attack generators. Deployment requires careful objective design and human oversight (R4, R5).

7.5 Ethical Considerations and Broader Impact

Dual-use risk: AITL architectures designed for safety testing could, if repurposed, autonomously generate attacks at scale. The same feedback loop that discovers defenses can discover exploits. Responsible deployment requires: (a) access controls on attack generators, (b) rate limiting on AITL loops, and (c) human approval gates for high-consequence actions.

Automation and labor displacement: AITL reduces demand for routine evaluation labor (red-teaming, QA testing, manual benchmarking). While this enables scale, it also displaces human evaluators. Organizations adopting AITL should consider reskilling pathways and retain human oversight roles.

Evaluator bias propagation: When AI judges replace human judges, biases encoded in the evaluator model propagate through the entire feedback loop without the corrective diversity that human judgment panels provide [Wang et al., 2023b]. Regular auditing of evaluator behavior (R4) is essential.

Accountability gap: In HITL systems, a human is accountable for each decision. In AITL, accountability diffuses across the loop. Establishing clear responsibility for AITL system outcomes remains an open governance question.

8 Conclusion

We proposed AI In The Loop (AITL) as a unifying framework for closed-loop autonomous systems, extending the RLAIF principle from training to the full AI system lifecycle.

Through analysis of AlphaZero, Constitutional AI, SWE-agent, and autoresearch, we identified common operational properties: self-generation, self-evaluation, self-improvement, and human observation. We formalized these into a taxonomy with explicit requirements (R1–R5) and failure modes (F1–F6).

Our AEOS experiment—the Autonomous Empirical Optimization System built to study constrained autonomous closed-loop evaluation—demonstrated two distinct agent behaviors on a semantically-stripped tabular classification task (Cover Type, 54 features, 7 classes, 10,000 samples):

- A local Qwen2.5-Coder-7B agent pragmatically converged on RandomForest, self-terminated in 9 iterations, and achieved 80.65% accuracy at $0 cost.
- A cloud GPT-4o-mini agent found a marginally better peak of 80.90% but continued low-yield exploration for 36 iterations, expending compute on MLP variants and ensemble combinations beyond the observed performance plateau—an instance of Sunk-Cost Continuation (F6).

In these experiments, the dominant human role shifts from iterative ML engineering toward **constraint management**: defining budgets, setting stagnation limits, and monitoring aggregate metrics ($O(1)$ per experiment, not $O(n)$ per iteration).

AITL suggests scalable directions for evaluation that are infeasible under HITL constraints. However, it introduces new failure modes—Sunk-Cost Continuation (F6), Evaluator Collapse (F2), and Objective Misspecification (F1)—requiring careful safety design and continued human oversight as system managers.

As AI systems deploy continuously and evaluation demands grow, AITL architectures like AEOS offer a path toward scalable, autonomous evaluation pipelines. The degree to which human oversight can be safely reduced—and the precise boundaries of fully autonomous closed-loop systems—remains an open question for future work.

A Reproducibility

A.1 Code Repository

The full experimental code is available at: https://github.com/m4vic/AITL

Key files:

- `experiments/aeos/data_loader.py`: Cover Type dataset logic
- `experiments/aeos/agent.py`: LLM agent with pivot & termination rules
- `experiments/aeos/trainer.py`: Sandboxed code execution
- `experiments/aeos/runner.py`: AITL orchestration loop
- `experiments/aeos/plot_advanced.py`: Step-frontier visualization

A.2 Reproduction Instructions

```
git clone https://github.com/m4vic/AITL.git
cd AITL/experiments/aeos
pip install torch torchvision scikit-learn matplotlib numpy openai

# Cloud Agent experiment (OpenAI)
export OPENAI_API_KEY="your-key-here"
python runner.py --backend openai --model gpt-4o-mini

# Local Agent experiment (Ollama)
# Ensure Ollama is running with qwen2.5-coder:7b
python runner.py --backend ollama --model qwen2.5-coder:7b
```

A.3 Full Experiment Log

Complete iteration-by-iteration results (all architectures, their code, loss, accuracy, and epochs run) are available in the repository under `results/run_*.json`.

References

Silver, D., Hubert, T., Schrittwieser, J., et al. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

Christiano, P. F., Leike, J., Brown, T., et al. Deep Reinforcement Learning from Human Preferences. *Advances in Neural Information Processing Systems*, 30, 2017.

Ouyang, L., Wu, J., Jiang, X., et al. Training Language Models to Follow Instructions with Human Feedback. *Advances in Neural Information Processing Systems*, 35, 2022.

Bai, Y., Kadavath, S., Kundu, S., et al. Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073*, 2022.

OpenAI. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*, 2023.

Karpathy, A. autoresearch: A framework for AI agents to conduct ML research autonomously. GitHub repository, accessed 2026. https://github.com/karpathy/autoresearch

Feurer, M., Klein, A., Eggensperger, K., et al. Efficient and Robust Automated Machine Learning. *Advances in Neural Information Processing Systems*, 28, 2015.

Olson, R. S., Bartley, N., Urbanowicz, R. J., & Moore, J. H. Evaluation of a Tree-based Pipeline Optimization Tool for Automating Data Science. *GECCO '16*, 2016.

Liu, H., Simonyan, K., & Yang, Y. DARTS: Differentiable Architecture Search. *International Conference on Learning Representations (ICLR)*, 2019.

Pham, H., Guan, M., Zoph, B., Le, Q., & Dean, J. Efficient Neural Architecture Search via Parameter Sharing. *International Conference on Machine Learning (ICML)*, 80, 2018.

Finn, C., Abbeel, P., & Levine, S. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. *International Conference on Machine Learning (ICML)*, 70, 2017.

Settles, B. Active Learning Literature Survey. *Computer Sciences Technical Report 1648*, University of Wisconsin–Madison, 2009.

Richards, T. AutoGPT: An Autonomous GPT-4 Experiment. GitHub repository, 2023. https://github.com/Significant-Gravitas/AutoGPT

Nakajima, Y. BabyAGI. GitHub repository, 2023. https://github.com/yoheinakajima/babyagi

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations (ICLR)*, 2024.

Zheng, L., Chiang, W.-L., Sheng, Y., et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36, 2023.

Wang, G., Xie, Y., Jiang, Y., et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*, 2023.

Wang, P., Li, L., Chen, L., et al. Large Language Models are not Fair Evaluators. *arXiv preprint arXiv:2305.17926*, 2023.

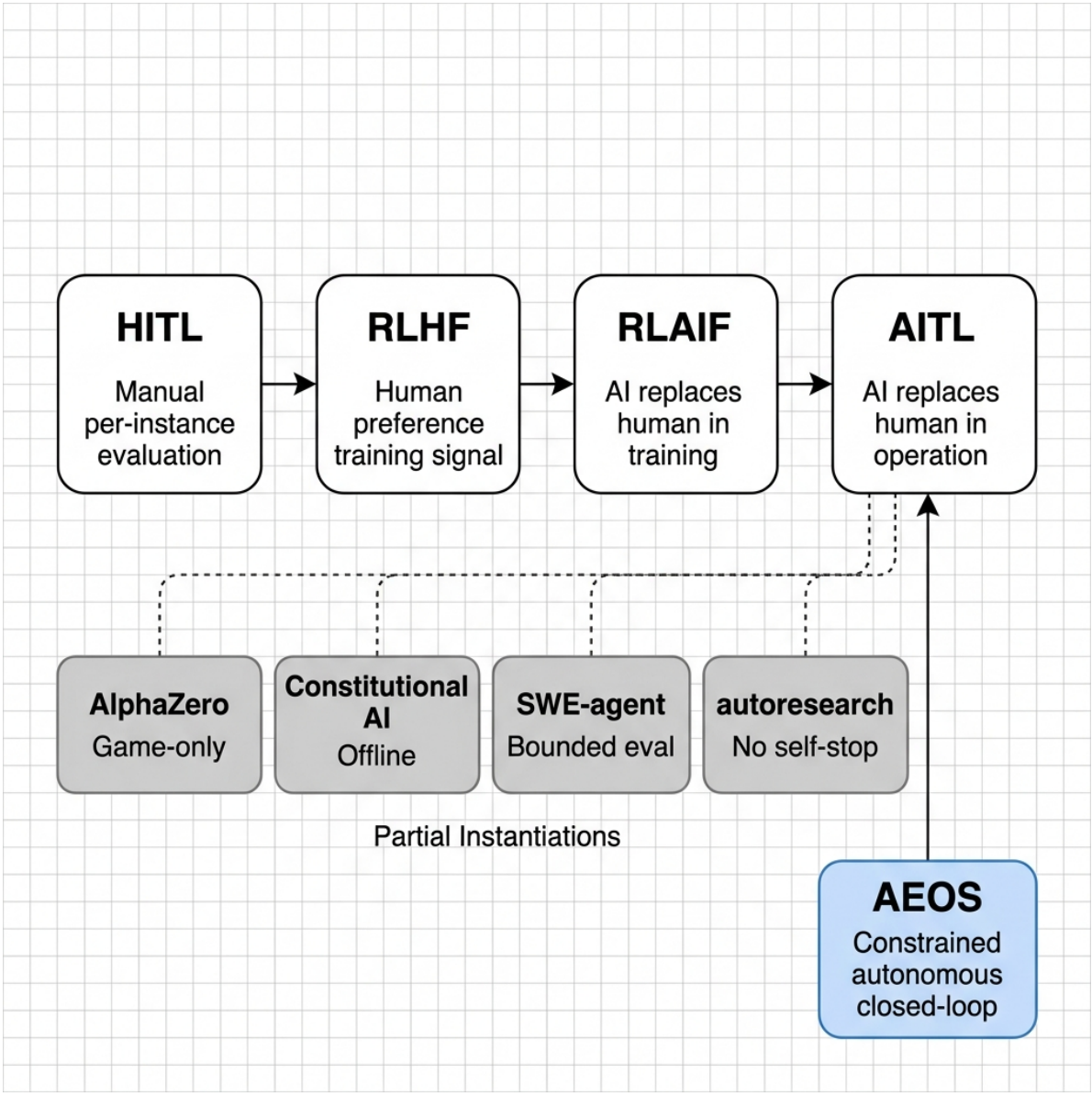

Figure 1: Evolution of AI evaluation paradigms: from Human-in-the-Loop (HITL) through RLHF and RLAIF to AI-in-the-Loop (AITL). Existing systems represent bounded instantiations; AEOS is our constrained autonomous closed-loop instance.

```
+-------------------------------------------------------------+
| AITL OUTER LOOP (Model Space Search) |
| |
| +----------+ +--------------+ +------------------+ |
| | Agent |--->| Trainer |--->| Feedback Logger | |
| |(LLM Code | | (Sandboxed) | | (JSON + Plot) | |
| | Writer) | | Exec. 120s | | | |
| +----------+ +--------------+ +--------+---------+ |
| ^ | |
| +---------------------------------------+ |
| Validation Metrics Feedback |
| |
| Agent dynamically rules: IF no further improvement |
| expected, output "STOP" to terminate the loop. |
+-------------------------------------------------------------+
```

Figure 2: AEOS loop architecture. The agent writes model training code, receives validation metrics as feedback, and must autonomously decide when to stop.

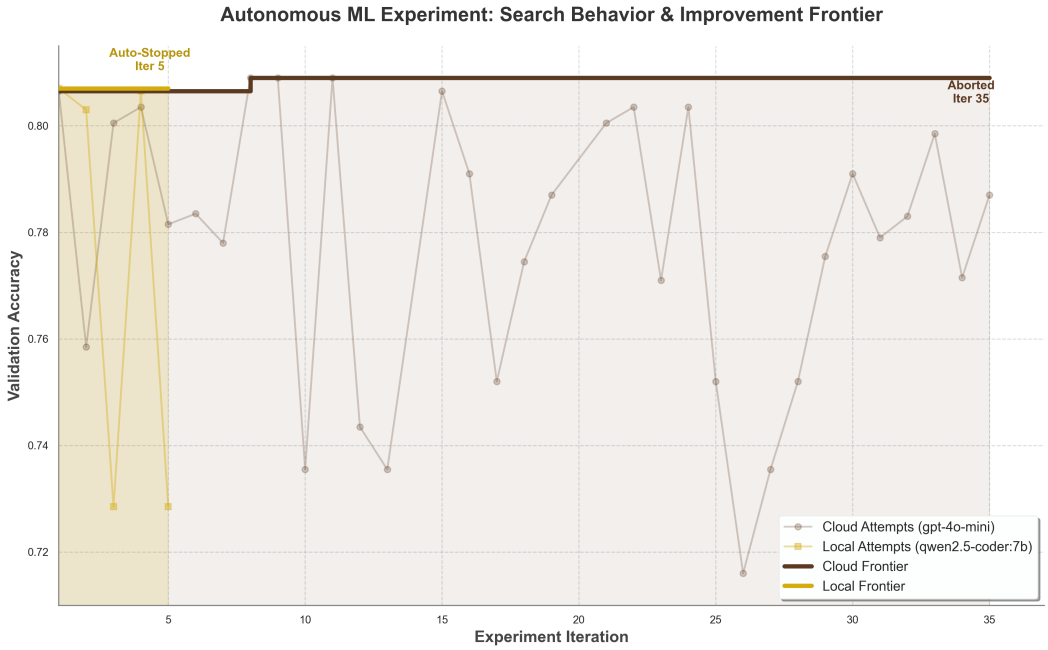

Figure 3: Comparative Best-So-Far discovery frontier between the two agents. The local model self-terminated upon reaching a performance plateau (9 iterations), while the cloud model continued exploration without emitting a stop signal for 28 additional iterations beyond its peak.